高效大语言模型

第 10 章（下）

第 16 课时 本课时讲解 10.3–10.6

推理特点 · Flash Attention · vLLM · 量化与投机解码

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 16 课时

10.1 LLM推理特点分析 第 3 页
推理两阶段与加速方案分类

10.2 Flash Attention 第 11 页
Online-Softmax 与分块注意力

10.3 vLLM 推理服务 第 62 页
分页注意力与连续批处理

10.4 量化压缩部署 第 77 页
SmoothQuant 与 AWQ

10.5 投机解码 第 86 页
块解码与投机解码

10.6 小结 第 97 页
四条加速路线各自的瓶颈与手段

本课时内容 10.3 vLLM 推理服务、10.4 量化压缩部署、10.5 投机解码、10.6 小结（其余部分见第 15 课时）

10.3

vLLM 推理服务

解码是存储密集型任务：能同时解码多少请求，取决于 KV-Cache 管得多细。vLLM 把操作系统的分页机制搬进了显存。

本节内容

10.3.1 推理服务问题

10.3.2 分页注意力机制

10.3.3 连续批处理

10.3.4 解码策略

推理服务的场景特点

大语言模型部署涉及嵌入式设备与云计算服务。云端通常要在 GPU 集群上对外提供服务，需针对 Transformer 的推理特性设计专用系统。

预填充任务

计算密集型

整条提示词一次前向，矩阵运算充分并行，决定首词元延迟。

解码任务

存储密集型

每步只处理一个词元，瓶颈在 KV-Cache 的读取与容量。

低延时 + 高吞吐

在有限资源下同时做到尽可能快的响应与尽可能大的批量。

解码批次量是关键

解码阶段串行推进，有限存储下能塞进多少请求决定吞吐。

两阶段可分离

按计算特性把预填充与解码分开部署，提高整体系统效率。

依据 《大语言模型：训练、精调与对齐》第 10.3 节

显存分配的碎片问题

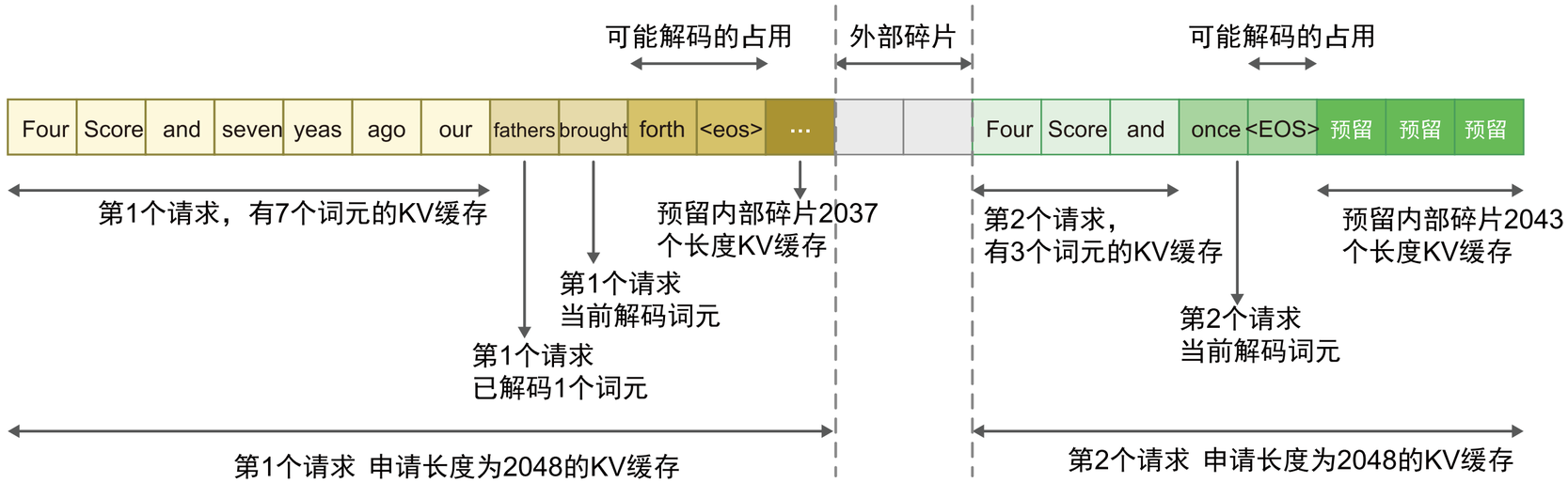


图 10.14　LLM 推理的 KV-Cache 存储碎片示例

内部碎片

为每个请求预留固定长度空间，解码提前终止后，预留部分被浪费。

外部碎片

不同请求的 KV-Cache 之间的间隙无法被其他请求使用。

两种浪费同时放大

用户提示词长度不确定、到达随机。

图 10.14 中，第一条请求含 7 个词元，额外预留 2048 词元空间，却只生成 4 个词元即终止；第二条请求同样预留固定长度，两条之间的间隙成为外部碎片。

图源 《大语言模型：训练、精调与对齐》图 10.14

键值缓存的显存占用

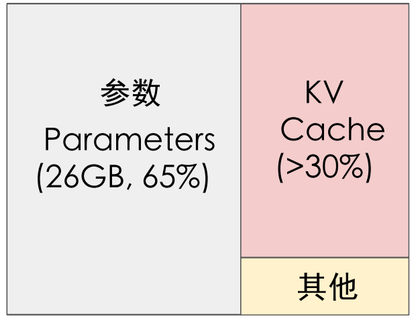


图 10.15(a)　显存占用分布

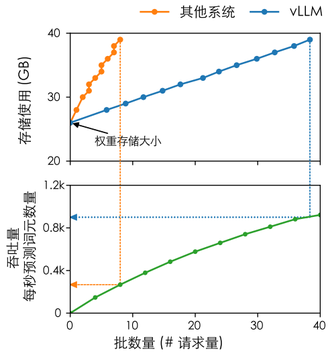


图 10.15(b)　vLLM 推理效率

图 10.15　KV-Cache 利用率和吞吐量之间的关系

- 以 A100（40GB）为例，KV-Cache 占用超过 30% 显存。
- 受限于存储墙，传统系统难以提升批量大小，进而限制推理速度。
- 高效管理 KV-Cache 可支持更大批量：vLLM 的词元生成速率较现有系统提升约 3 倍，可支撑的解码批量显著更大。

图源　《大语言模型：训练、精调与对齐》图 10.15(a)(b)

显存利用率的差距

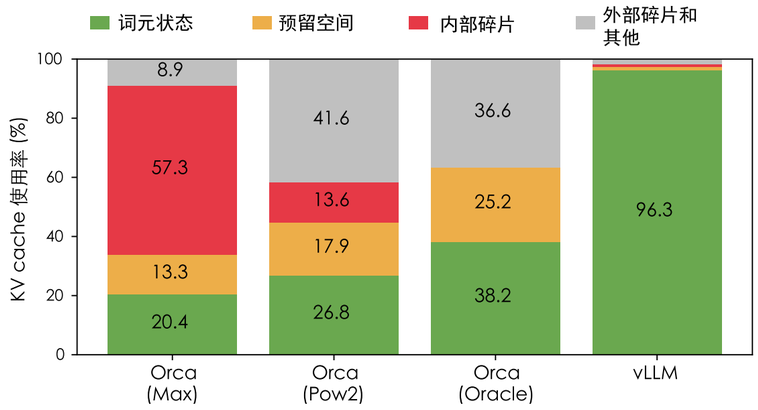


图 10.16　vLLM 系统的 KV-Cache 管理效率

38.2%

Orca 的 KV-Cache 利用率

≈100%

vLLM 的利用率，接近零浪费

- 传统管理策略在“预留”与“对齐”上浪费了大量可用空间。
- vLLM 让几乎每一个显存页都能存放有效 KV-Cache。

图源 《大语言模型：训练、精调与对齐》图 10.16

三种显存管理策略

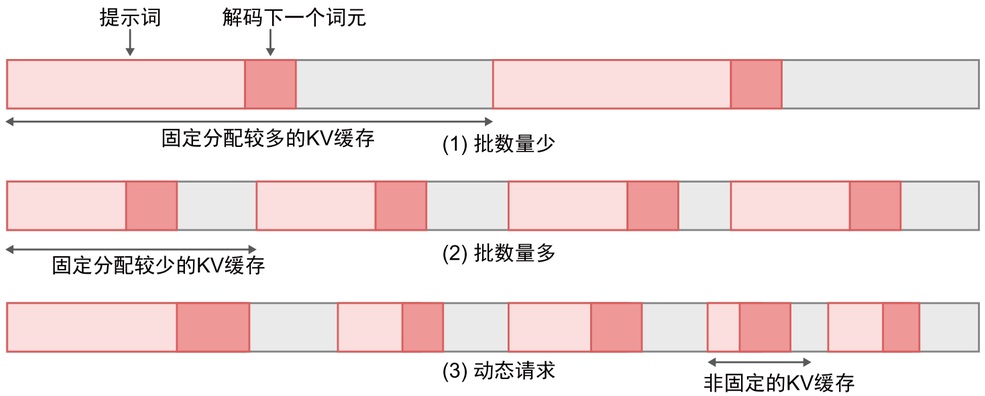


图 10.17　分块 KV-Cache 管理系统

策略	后果
(a) 固定大块	块内用不满，产生碎片
(b) 固定小块	限制单请求可处理的长度
(c) 分块管理	动态块分配，请求可跨越多个**非连续**存储块

关键差别：分块管理允许一个请求的 KV-Cache 不连续存放，因此不必为“最长可能序列”预留连续空间。

图源 《大语言模型：训练、精调与对齐》图 10.17

显存管理策略的对比

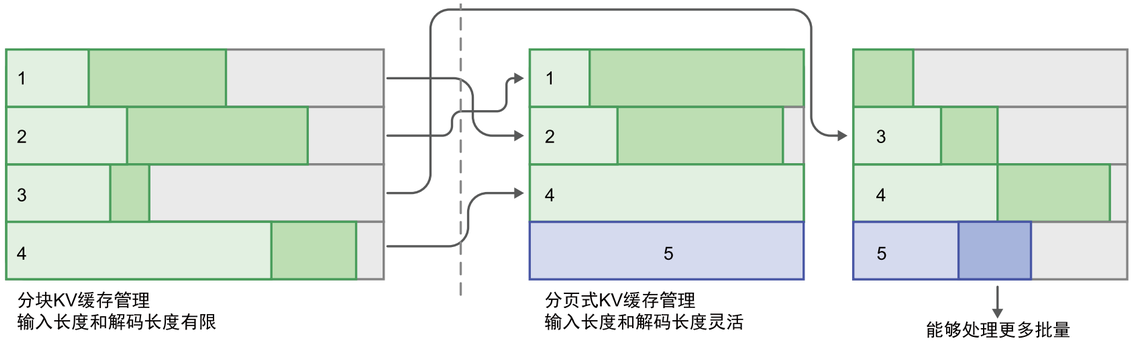


图 10.18　灵活的 KV-Cache 存储管理提高处理请求数量

- 原始 4 个存储块只支持 4 个请求。
- 分块管理后，请求 2 和 4 可以跨块存储。
- 新增的请求 5 可利用空闲块，同样存储大小下多承载一个请求。
- 块与块之间不要求连续。

图源 《大语言模型：训练、精调与对齐》图 10.18

页式显存管理

vLLM 借鉴操作系统的分页式存储管理：将物理显存划分为固定大小的页，并建立虚拟地址到物理地址的映射关系。

请求到达
序列长度未知

→

分配虚拟页
不要求物理连续

→

块表
虚拟页 → 物理页

→

填满即追加
动态分配新物理页

- 避免了为每个请求预先分配连续存储空间的需求。
- 序列长度超过单页容量时，虚拟内存管理会动态分配新的物理页。
- 同一请求的 KV-Cache 因而可以分布在非连续的物理存储空间中。

vLLM 的系统组成

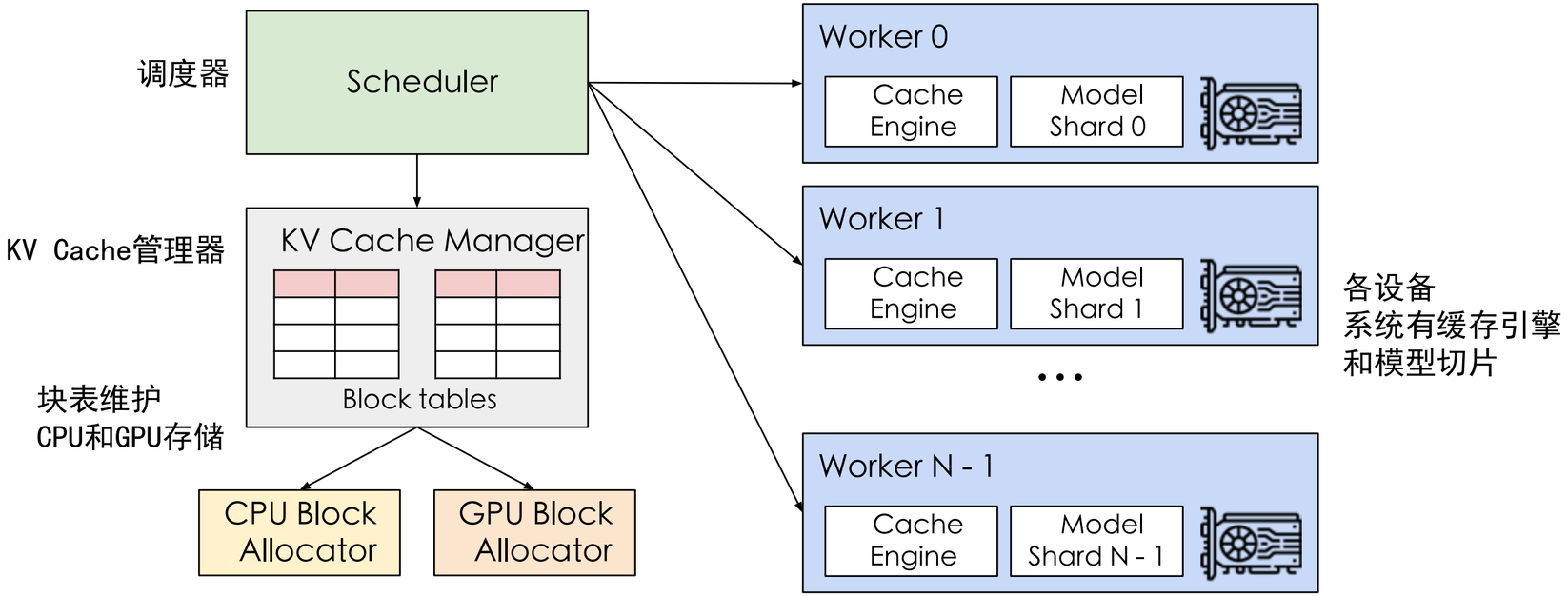


图 10.19　vLLM 逻辑架构图

调度器

决定哪些请求进入当前批次、何时抢占与释放页。

KV-Cache 管理器

通过块表维护虚拟页与物理页的映射关系。

工作器

执行单元，配备独立的缓存引擎与分片模型参数。

图源 《大语言模型：训练、精调与对齐》图 10.19

PagedAttention

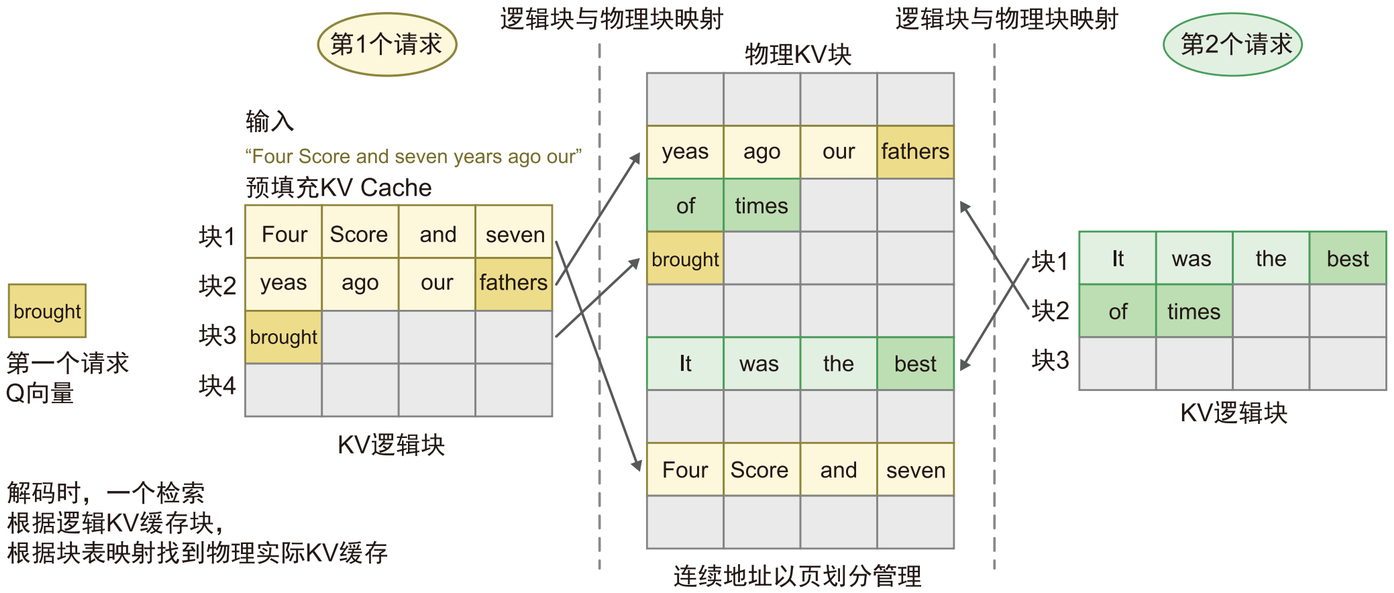


图 10.20　虚拟页管理系统处理两个请求示例

- 解码时查询向量 $\boldsymbol{Q}$ 与分页存放的 KV-Cache 计算注意力。
- 分页的 KV-Cache 与 Flash Attention 对 KV 分块处理的形式类似，可直接复用其计算内核。
- 页面填满时，块表自动建立当前页到新物理页的映射。
- 图 10.20 中请求 2 的 KV-Cache 跨越两个物理块；多请求下调页机制可灵活分配空闲页。

理论保证　分页注意力与标准注意力的计算结果一致——系统只改变了 KV-Cache 的存储管理方式。

图源　《大语言模型：训练、精调与对齐》图 10.20

静态批处理的空转

一批请求的输入长度与解码长度各不相同。部分请求提前结束解码，而其他请求仍在继续。传统批处理必须等整批全部完成，提前结束的请求所占用资源只能空转。

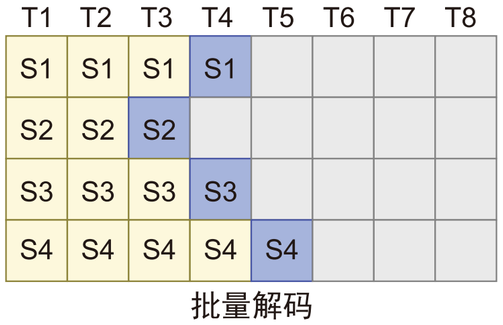


图 10.21(a)　批量解码

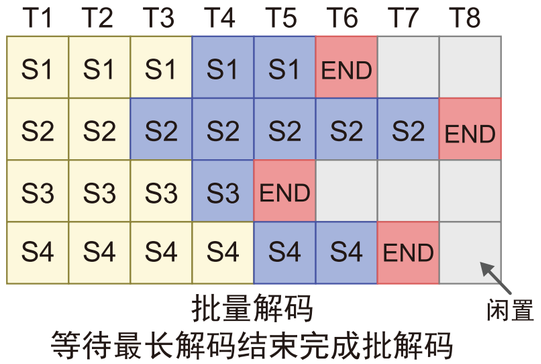


图 10.21(b)　静态批量解码

图 10.21　连续批处理示例

图 (a)：系统先预填充黄色区域的 KV-Cache，并生成首个解码词元（蓝色）。图 (b)：部分请求提前终止，静态批量解码仍需等待整批完成。

图源 《大语言模型：训练、精调与对齐》图 10.21(a)(b)

连续批处理

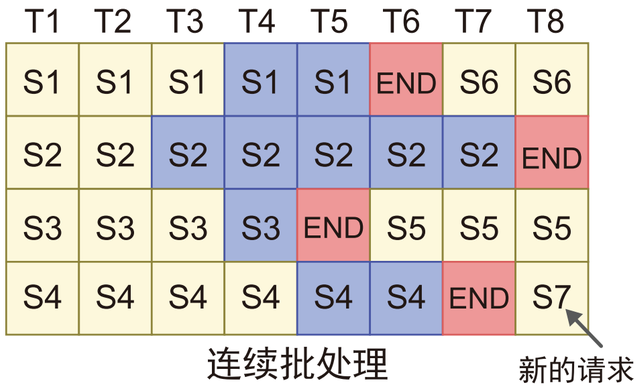


图 10.21(c)　连续批处理

连续批处理通过动态调度，在空闲资源上分配新请求的 KV-Cache，实现持续的词元预测。

- 终止请求释放的存储空间，立即填入新请求的 KV-Cache。
- 调度与解码持续进行，不必等待整批同步。
- 结合虚拟页管理，可同时处理更多请求。
- 遇到长序列解码时，动态资源调配避免空闲等待，实现实时响应。

图源 《大语言模型：训练、精调与对齐》图 10.21(c)

前缀共享

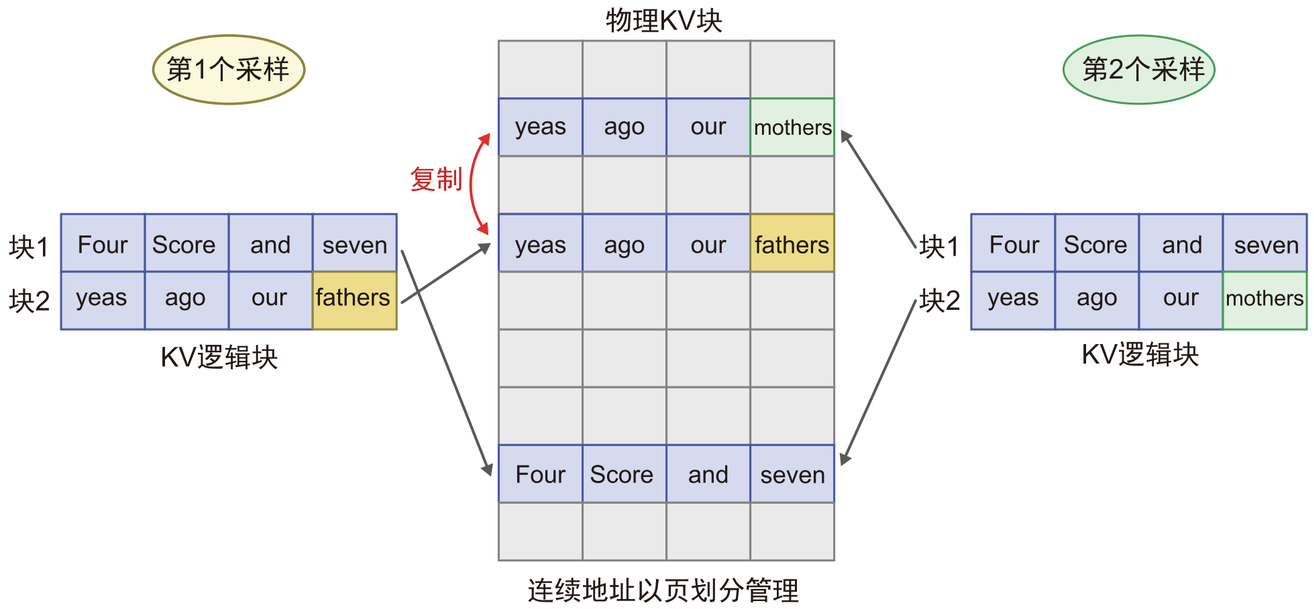


图 10.22　共享前缀解码机制

- 拒绝采样、束搜索等场景会从一个前缀分出多个解码分支。
- 分页注意力支持共享前缀：各分支共用前缀部分的物理页。
- 生成新分支时通过复制操作创建新存储块，使各分支独立解码而不干扰共享前缀。

图源 《大语言模型：训练、精调与对齐》图 10.22

束搜索中的写时复制

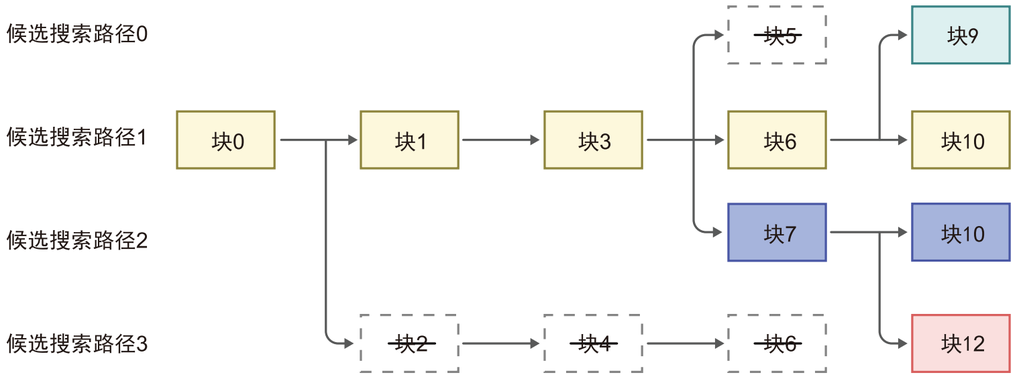


图 10.23　束搜索中的共享前缀应用

- 束搜索同时维护多条候选序列，前缀高度重复。
- 共享前缀把重复部分压缩成一份物理页，只在新分支处分叉。
- 长文本推理中还可共享系统提示词或上下文提示词的 KV-Cache，避免重复预填充。

> **收益**　显存开销显著降低，同时省掉重复的预填充计算。

图源 《大语言模型：训练、精调与对齐》图 10.23

vLLM 的三项技术

分页注意力

消除碎片

物理页 + 块表，请求可跨非连续块；利用率从 38.2% 提升到接近零浪费。

连续批处理

提高批量

请求结束即刻释放页并接纳新请求，解码不再等整批同步。

共享前缀

减少冗余

公共前缀只存一份，分支以复制创建新块，独立推进。

- 三者叠加，使解码批量与吞吐量同时提升，而注意力的数学结果保持不变。
- 分页 KV-Cache 的分块形式与 Flash Attention 的 KV 分块一致，计算内核可以直接复用。

10.4

量化压缩部署

权重好量化，激活值难量化。两种方法各自动刀：一个用等价变换把难度搬给权重，一个用搜索找出该保护的通道。

本节内容

10.4.1 SmoothQuant

10.4.2 AWQ

量化的精度格式

大语言模型参数量极大，推理时通常以 Float16 或 BFloat16 精度加载。量化能在更低比特下存储与计算。

权重 8 位 / 激活 8 位

W8A8

权重与激活值都量化，可用低比特矩阵乘。SmoothQuant 支持这一形式。

权重 4 位 / 激活 16 位

W4A16

只量化权重，适配不支持 W8A8 算子的硬件，适合端侧部署。

权重 4 位 / 激活 8 位

W4A8

权重压得更狠，激活保留 8 位，兼顾显存与精度。

- 按数据类型不同，量化方案可表示为 W8A8、W4A8、W4A16 等形式。
- 其他方法还包括 GPTQ、LLM-int4 等。本章重点读 SmoothQuant 与 AWQ。

激活值的离群通道

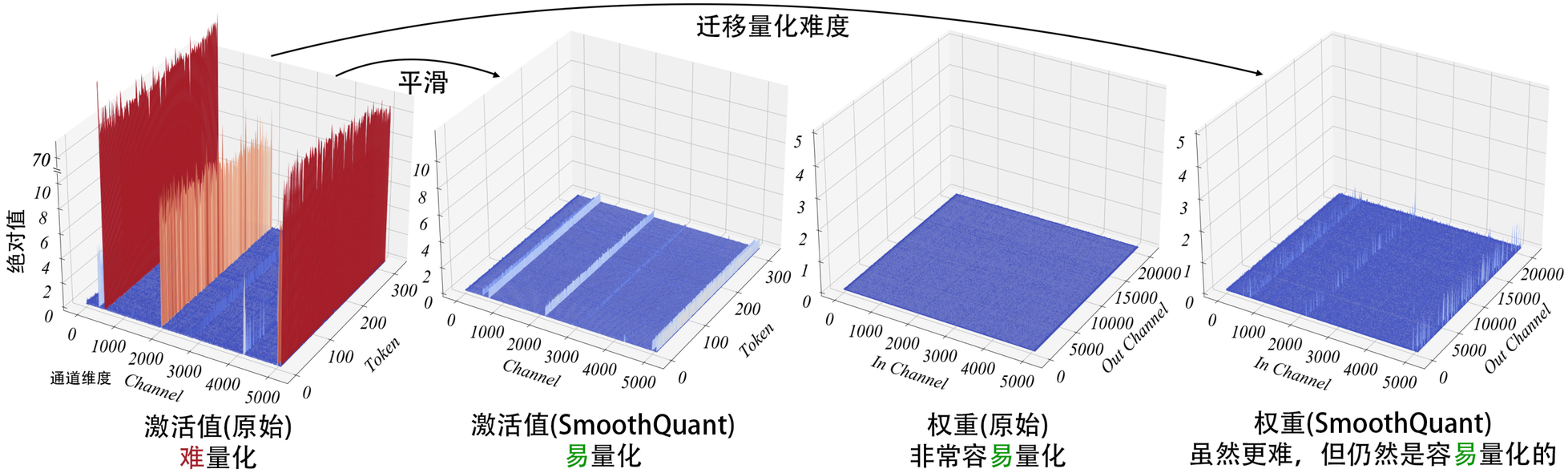


图 10.24　激活值的特征通道离群值分析图

权重均匀

权重分布通常较均匀，易于量化。

激活离群

激活值在特定特征通道上出现显著峰值（离群点）。

尺度难选

离群点增大了数值方差，使以张量块为单位的量化难以选出一个合适的缩放尺度。

SmoothQuant 是一种无须训练、保持精度的量化算法，支持对权重和激活值同时量化。

图源 《大语言模型：训练、精调与对齐》图 10.24

平滑量化

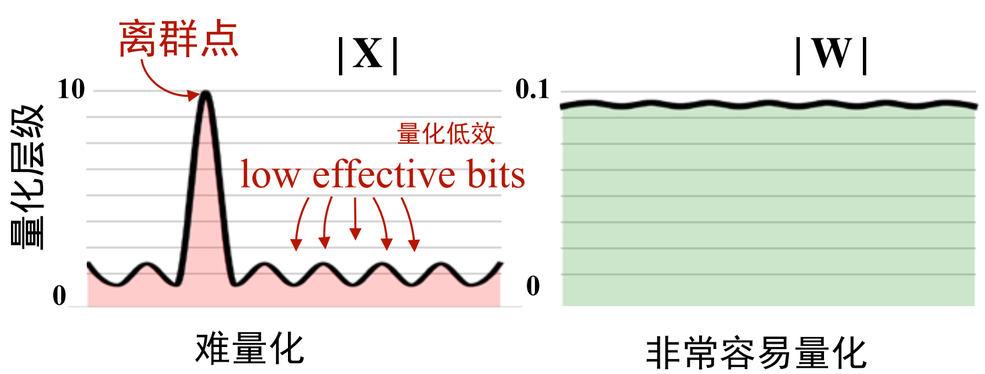


图 10.25(a)　原始

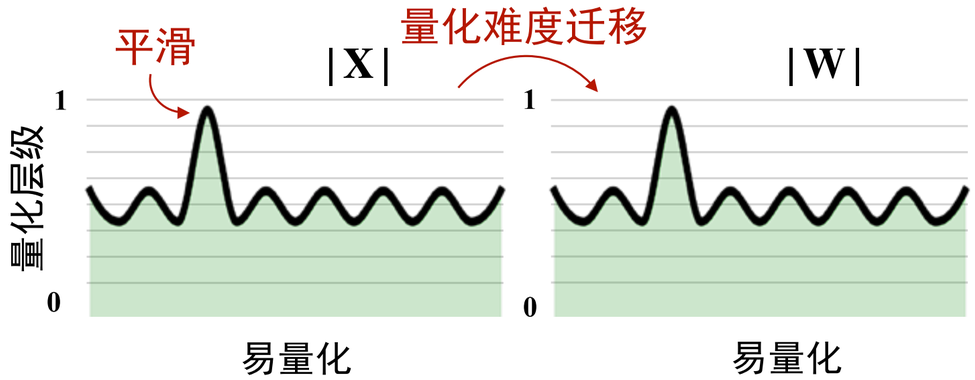


图 10.25(b)　SmoothQuant

图 10.25　SmoothQuant 量化示意图

- 对激活值做平滑，同时对权重做反向缩放，保持输出等价。
- 平滑后离群点的影响降低，量化难度显著减小。
- 本质是把激活值的量化复杂度迁移到权重，而权重量化对此类迁移不敏感。

图源 《大语言模型：训练、精调与对齐》图 10.25(a)(b)

按特征通道缩放

$$\boldsymbol{Y} = (\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}) \cdot (\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}) = \hat{\boldsymbol{X}}\hat{\boldsymbol{W}} \tag{10-91}$$

$\boldsymbol{s} \in \mathbb{R}^d$ 为自适应缩放因子，d 为特征维度。先用 $\mathrm{diag}(\cdot)$ 把 $\boldsymbol{s}$ 变成对角矩阵：$\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}$ 逐特征列缩小，$\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}$ 逐特征行放大，两者相乘仍等于 $\boldsymbol{X}\boldsymbol{W}$。

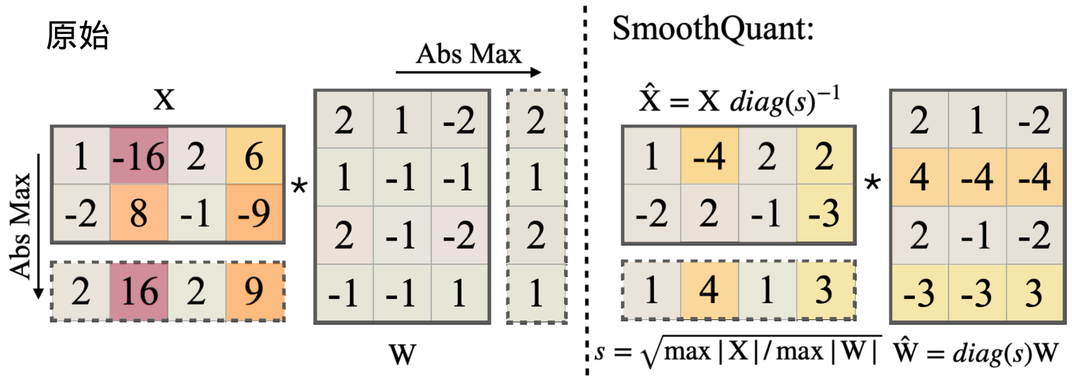


图 10.26　特征通道计算独立的缩放因子

- 量化在激活值的**特征通道维度**上执行，而非张量块级。
- （1）用 $\mathrm{diag}(\cdot)$ 把 $\boldsymbol{s}$ 转为对角矩阵，与激活值相乘即逐列缩放。
- （2）对激活值与权重分别做逆缩放与缩放，输出等价。

图源 《大语言模型：训练、精调与对齐》图 10.26；式 (10-91)

缩放因子的计算

$$\boldsymbol{s} = \sqrt{\mathrm{colmax}(\boldsymbol{X}) / \mathrm{rowmax}(\boldsymbol{W})} \tag{10-92}$$

对激活值按列（特征通道）取最大值 $\mathrm{colmax}(\boldsymbol{X})$，对参数按行取最大值 $\mathrm{rowmax}(\boldsymbol{W})$，两者相除再开方。

激活离群大

$\mathrm{colmax}(\boldsymbol{X})$ 大 → s_j 大 → 该通道被除得更多。

权重该行大

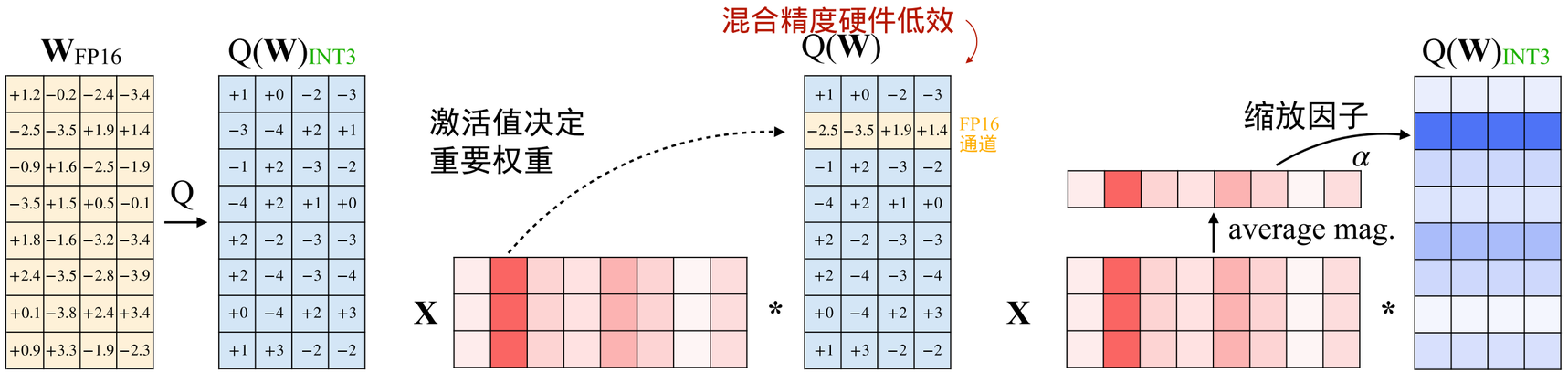
$\mathrm{rowmax}(\boldsymbol{W})$ 大 → s_j 小 → 权重不再被继续放大。

开方

把难度在激活与权重之间对半分摊。

SmoothQuant 使用校准样本从预训练数据集估计激活通道的范围。缩放因子离线统计得到，而非在推理时实时计算。

关键通道的作用



(a) 舍入量化 (**PPL 43.2**)　(b) 保留1% FP16 权重 (**PPL 13.0**)　(c) 量化前缩放权重(**PPL 13.0**)

图 10.27　AWQ 量化原理图

只有少数通道敏感

权重中存在对量化精度敏感的**重要通道**；仅对约 1% 的关键通道保持原始精度，即可显著提升整体量化精度。

因此选择低比特

少量通道的数值对量化误差极为敏感，AWQ 因此专注 3-bit、4-bit 的低比特方案。

> **部署收益**　部分硬件不支持 W8A8 矩阵乘，W4A16（权重 int4、激活 fp16）可适配更多平台；低比特权重量化同时显著降低存储与计算需求，适合资源受限的设备端。

图源 《大语言模型：训练、精调与对齐》图 10.27

平滑量化的三个步骤

1　缩放权重

$\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}$

→

2　量化权重

$Q(\mathrm{diag}(\boldsymbol{s})\boldsymbol{W})$

→

3　逆缩放激活

$\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}$

$$\mathcal{L}(\boldsymbol{s}) = \left\| \left(\mathrm{diag}(\boldsymbol{s})^{-1}\boldsymbol{X}\right)\left(Q(\boldsymbol{W}\mathrm{diag}(\boldsymbol{s}))\right) - \boldsymbol{X}\boldsymbol{W} \right\| \tag{10-93}$$

为保持原始输出精度，最小化"量化权重 × 缩放激活"与原始输出 $\boldsymbol{X}\boldsymbol{W}$ 的均方误差。$Q(\cdot)$ 为包含就近舍入（round-to-nearest）的整数量化函数。

$$\boldsymbol{s}^{*} = \arg\min_{\boldsymbol{s}} \mathcal{L}(\boldsymbol{s}) \tag{10-94}$$

目标是找到使损失最小的通道级缩放因子。由于就近舍入操作不可导，AWQ 采用网格搜索求次优解，而不是梯度下降。

依据 《大语言模型：训练、精调与对齐》第 10.4.2 节（式 (10-93)、(10-94)）

两种量化思路的对比

对比项	SmoothQuant	AWQ
量化对象	权重 + 激活值（W8A8）	主要量化权重（W4A16 等）
核心观察	激活值存在通道级离群点	约 1% 通道对量化误差极敏感
缩放因子来源	离线统计：$\sqrt{\mathrm{colmax}(\boldsymbol{X})/\mathrm{rowmax}(\boldsymbol{W})}$	优化求解：最小化量化前后输出误差
求解方式	闭式公式，无需训练	网格搜索（舍入不可导）
主要收益	激活值也可 8 位，启用低比特矩阵乘	权重压到 4-bit，适配更多硬件

共同点 两者都用“缩放 + 逆缩放”的等价变换：不改变数学结果，只改变数值分布，使量化误差变小。

依据 《大语言模型：训练、精调与对齐》第 10.4.1、10.4.2 节

10.5

投机解码

解码一次只能出一个词元。若先用低成本方式猜出若干候选，再让目标模型一次并行验证，串行步数就能成倍减少。

本节内容

10.5.1 块解码

10.5.2 投机解码

并行解码的思路

串行解码每次只能生成单个词元。若能一次生成多个，解码效率即可显著提升——这类技术统称并行解码。

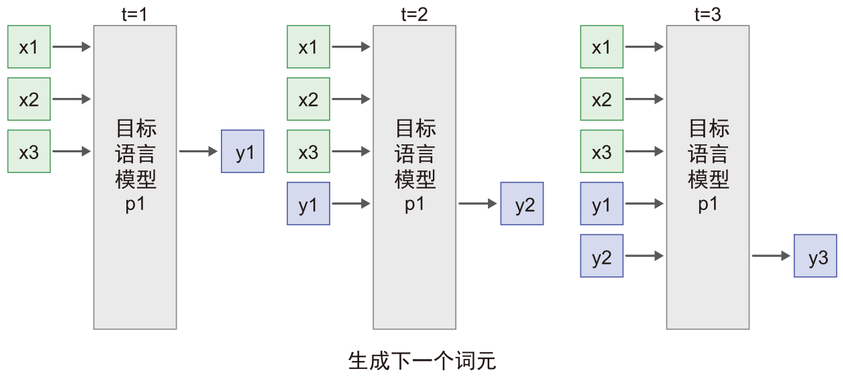


图 10.28(a)　草稿阶段：逐个生成词元

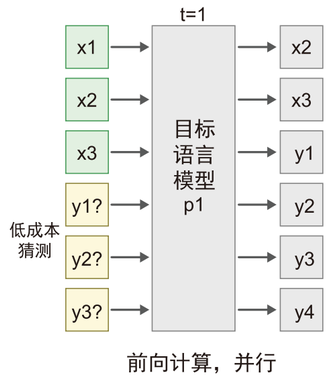


图 10.28(b)　验证阶段：并行验证候选

图 10.28　投机解码猜测与验证示意图

核心思想：先低成本猜测后续词元（黄色），再在目标模型上并行前向验证。

图源 《大语言模型：训练、精调与对齐》图 10.28(a)(b)

双模型推测架构

块并行解码采用双模型架构：目标解码模型 θ_1 预测下一词元，推测模型 θ_i 预测后续第 i 个词元。

$$\hat{y}_{j+i} = \arg\max_{y_{j+i}} p_{\theta_i}(y_{j+i} \mid \hat{\boldsymbol{y}}_{\leqslant j}, \boldsymbol{x})$$

预测阶段：推测模型 θ_i 在给定已有前缀 $\hat{\boldsymbol{y}}_{\leqslant j}$ 与输入 $\boldsymbol{x}$ 的条件下，给出位置 $j+i$ 的词元，$i = 1, 2, \cdots, k$。

$$\hat{y}_{j+i} = \arg\max_{y_{j+i}} p_{\theta_1}(y_{j+i} \mid \hat{\boldsymbol{y}}_{\leqslant j+i-1}, \boldsymbol{x})$$

验证阶段：目标模型 θ_1 以“到前一位置为止”的前缀为条件，逐位置检验预测是否正确。

预测　每个推测模型各管一个位置，互不依赖，因此可以并行生成整块候选。

验证　前 $\hat{k}$ 个词元逐个比对，通过即接受，第一个不符处终止；验证过程本身也是并行的。

目标模型 θ_1

给出真实分布，负责验证与最终采样。

推测模型 θ_i

低成本给出候选块，i 越大负责的位置越靠后。

接受

通过验证的词元作为本轮输出，失败则退回单步解码。

依据　《大语言模型：训练、精调与对齐》第 10.5.1 节（块解码流程）

预测阶段

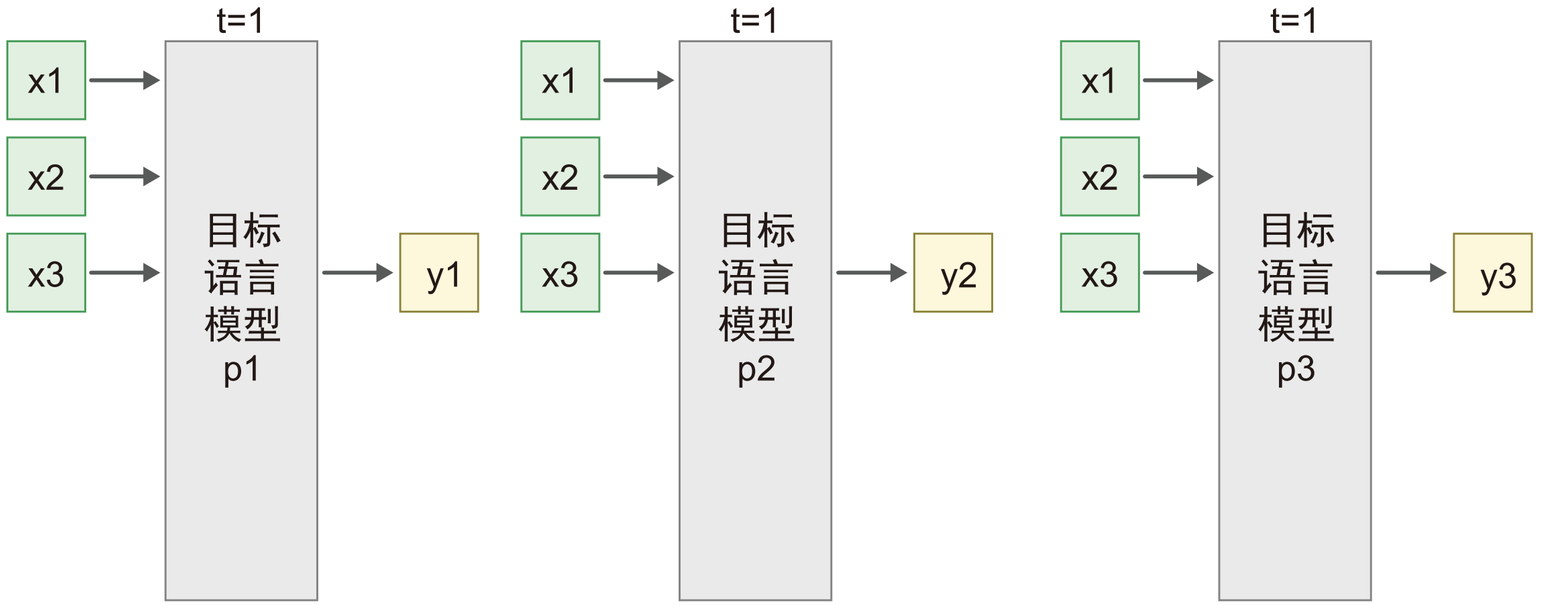


图 10.29　块并行解码流程示例预测阶段

- 第 i 个推测模型负责位置 $j+i$，各自独立。
- 同一轮内所有推测可并行执行，不必串行等待。
- 得到的是一条长度为 k 的候选块，而不是单个词元。

图源 《大语言模型：训练、精调与对齐》图 10.29

验证与接受

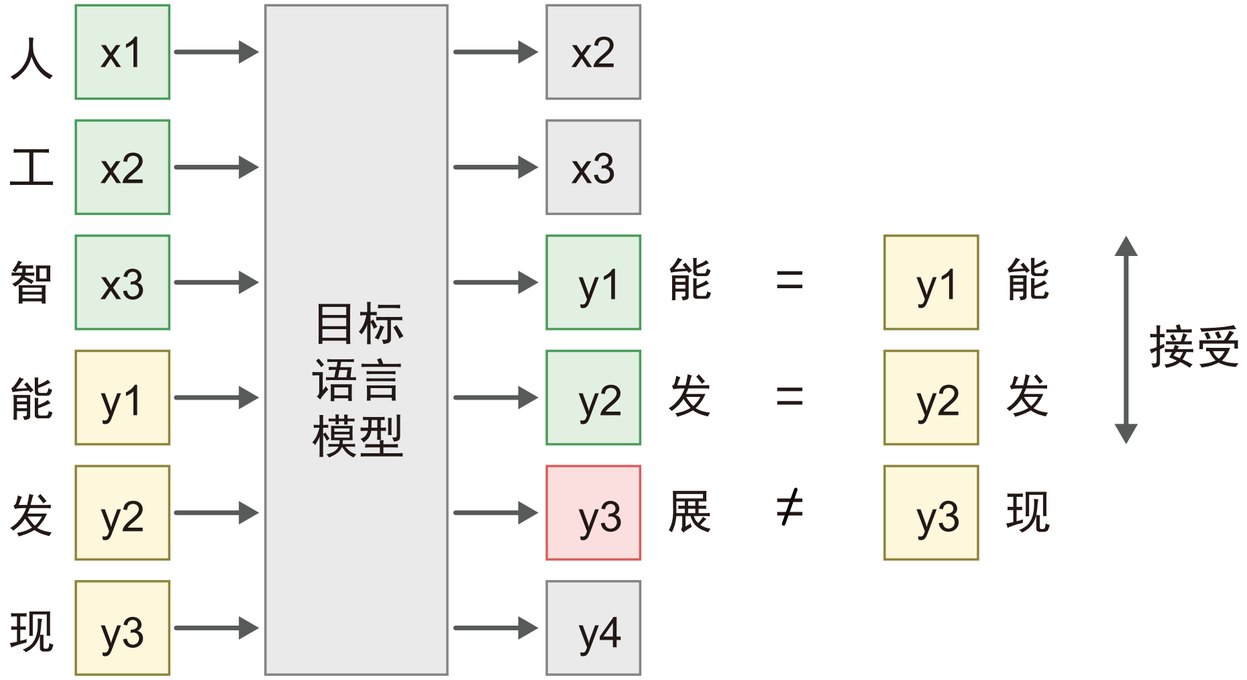


图 10.30　块并行解码流程示例验证阶段

- 验证失败时（如词元“现”），终止当前验证序列。
- 接受“能”“发”两个词元后，由 θ_1 生成新词元“展”。
- 单次迭代共解码 3 个词元。
- 若全部推测失败，系统退化为目标模型的单步解码。

接受 $\hat{k}$ 个词元后，θ_1 可继续预测位置 $y_{j+\hat{k}+1}$，进入下一轮。

图源 《大语言模型：训练、精调与对齐》图 10.30

验证与预测的并行

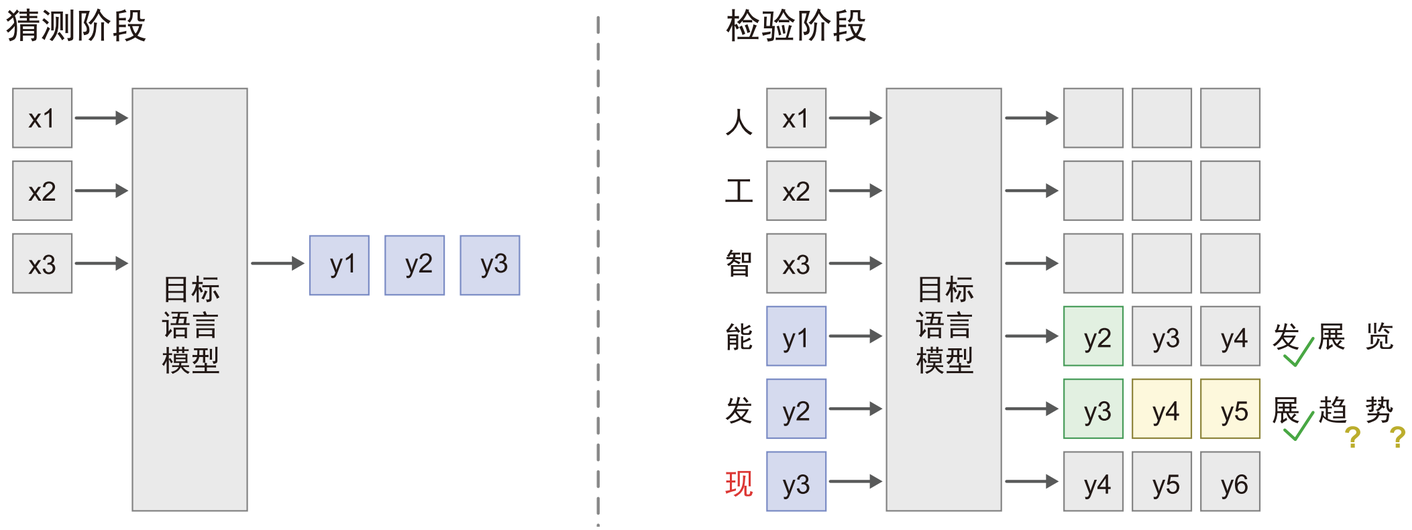


图 10.31　边验证边解码示例图

- 目标模型 p_{θ_1} 验证当前候选词元时，推测模型 p_{θ_i} 同步预测后续词元。
- 验证当前块（如 3 个词元）的同时生成下一候选块。
- 即使“现”验证失败，系统仍接受“展”；同步预测的“趋”“势”可直接用于下一验证周期。

> **收益**　验证与预测并行，失败也不再空等一轮，单次迭代的有效产出更高。

图源 《大语言模型：训练、精调与对齐》图 10.31

多输出头架构

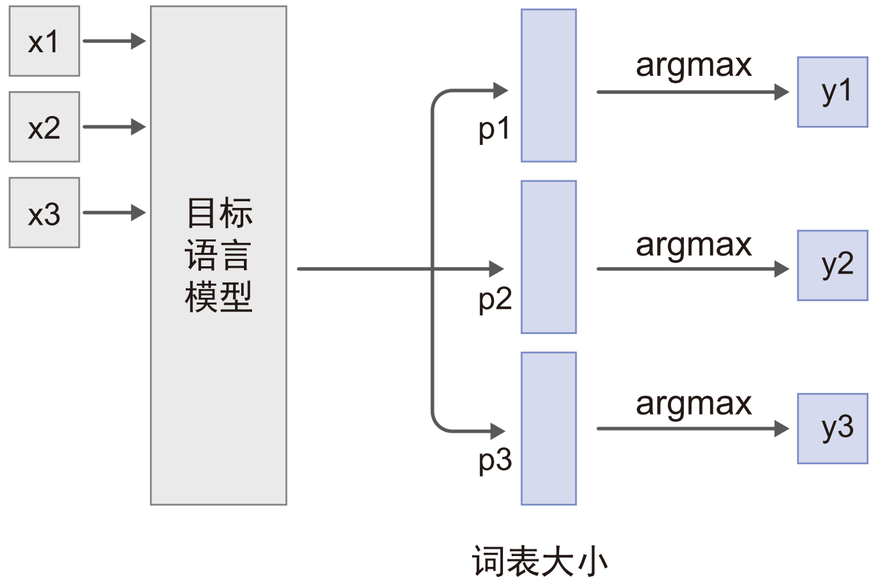


图 10.32　多输出头语言模型

验证时若要同步运行多个推测模型，开销不小。一种高效实现是把 Transformer 的单输出头改为多输出头结构。

- 第 i 个头负责预测后续第 i 个词元；$i = 1$ 时即常规的下词元预测。
- 各头具有差异化训练目标，但共享主干参数。
- 无需额外执行推测模型，单模型即可完成验证-预测并行，多头共享参数也提升计算效率。

Medusa、Eagle 系列等算法通过优化推测机制，显著提升了候选词元的接受率。

图源 《大语言模型：训练、精调与对齐》图 10.32

草稿模型与目标模型

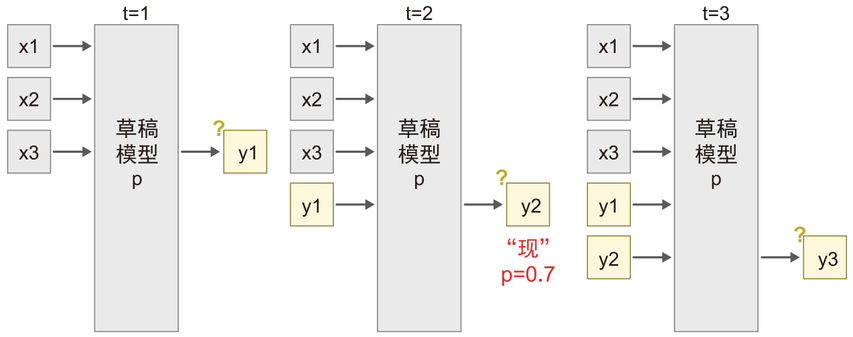


图 10.33(a)　草稿阶段

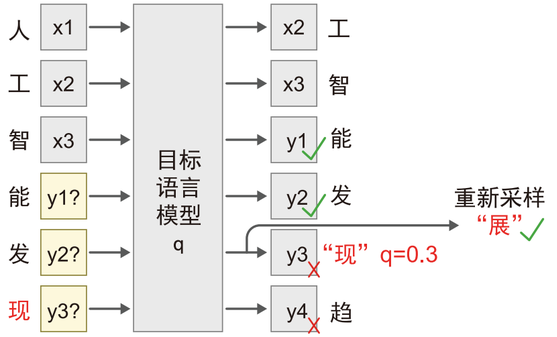


图 10.33(b)　验证阶段

图 10.33　投机解码示例

- 草稿模型尺寸更小、解码更快，输出分布接近目标模型，可通过知识蒸馏学习目标模型的分布。
- 草稿阶段一次预测 3 个词元；验证阶段目标模型逐位检验。
- “现”未通过检验，系统重采样得到“展”。

图源 《大语言模型：训练、精调与对齐》图 10.33(a)(b)

投机解码流程

$$\tilde{x}_t \sim p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t}), \qquad t = 1, 2, \cdots, k \tag{10-95}$$

第 1 步　草稿模型 p 自回归地解码出 k 个候选词元 $\tilde{\boldsymbol{x}} = (\tilde{x}_1, \cdots, \tilde{x}_k)$。

$$q(x|\boldsymbol{x}, \tilde{x}_t), \qquad t = 1, 2, \cdots, k \tag{10-96}$$

第 2 步　输入 $\boldsymbol{x}$ 与候选 $\tilde{\boldsymbol{x}}$ 拼接后送入目标模型 q，一次前向并行算出各位置的条件概率。

$$r < \min\left(1, \frac{q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})}{p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})}\right) \qquad t = 1, 2, \cdots, k \tag{10-97}$$

第 3 步　采样 $r \sim U[0, 1]$；不等式成立则接受 x_t。比值 q/p 越大越容易接受。

$$\tilde{x}_t \sim (q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t}) - p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})) \tag{10-98}$$

第 4 步 · 拒绝时　从两分布之差（取正部并归一化）重采样，并终止后续验证。

$$x_{k+1} \sim q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}) \tag{10-99}$$

第 5 步 · 全部接受时　由目标模型再采样一个词元，本轮推进 $k+1$ 个词元。

- 全部接受：$\boldsymbol{x} \leftarrow (\boldsymbol{x}, \tilde{\boldsymbol{x}}, x_{k+1})$；第 t 个被拒：$\boldsymbol{x} \leftarrow (\boldsymbol{x}, \tilde{\boldsymbol{x}}_{\leqslant t})$。
- 出现终止词元或达到目标长度则退出，否则回到第 1 步。
- 草稿阶段是 k 次串行小模型前向，验证阶段是 1 次大模型前向；一轮最少产出 1 个词元、最多 $k+1$ 个。

依据　《大语言模型：训练、精调与对齐》第 10.5.2 节（式 (10-95)–(10-99)）

等价性证明

$$\Pr(X=x)=\Pr(\tilde{x}=x)\Pr(\text{接受}|\tilde{x}=x)+\Pr(\text{拒绝})\Pr(X=x|\text{拒绝}) \tag{10-100}$$

$$\Pr(\tilde{x}=x)\Pr(\text{接受}|\tilde{x}=x)=p(x)\min\left(1,\tfrac{q(x)}{p(x)}\right) \tag{10-101}$$

$$=\min\left(p(x),q(x)\right) \tag{10-102}$$

$$\Pr(X=x|\tilde{x}\text{ 拒绝})=(q(x)-p(x))_+ \tag{10-103}$$

$$\Pr(\tilde{x}\text{ 拒绝})=1-\Pr(\tilde{x}\text{ 接受}) \tag{10-104}$$

$$=1-\sum_{x'}\min(p(x'),q(x')) \tag{10-105}$$

$$=\sum_{x'}\max(0,q(x')-p(x')) \tag{10-106}$$

$$\Pr(\tilde{x}\text{ 拒绝})\Pr(X=x|\tilde{x}\text{ 拒绝})=\max(0,q(x)-p(x)) \tag{10-107}$$

$$\Pr(X=x)=\min(p(x),q(x))+\max(0,q(x)-p(x)) \tag{10-108}$$

$$=q(x) \tag{10-109}$$

式 (10-100) 按“采到并被接受”“被拒后重采样”两种互斥情形拆开；式 (10-101) 代入接受概率 $\min(1,q/p)$，式 (10-102) 合并为 $\min(p,q)$；式 (10-103) 给出重采样的条件分布（取差的正部并归一化）；式 (10-105) 把接受事件按 x' 拆开求和，式 (10-106) 用 $1-\min(a,b)=\max(0,b-a)$；式 (10-107) 中归一化因子与式 (10-106) 的求和相同，相乘即约去；式 (10-108) 合流后两种情形都得到 $q(x)$。

结论 投机解码的输出分布与目标模型 q 完全一致，因此是“无损加速”，而不是用精度换速度。

块解码与投机解码

对比项	块并行解码	投机解码
候选来源	多个推测模型或多个输出头	一个更小的草稿模型
模型数量	θ_1 加 θ_i，或单模型多头	草稿模型 p + 目标模型 q
验证方式	θ_1 逐位置比对，遇错截断	按 q/p 的比值做随机接受检验
输出保证	接受即输出，拒绝则截断	分布与 q 严格等价
关键指标	接受长度 $\hat{k}$	接受率与草稿成本

两者的共同前提：验证时一次前向可以并行算出多个位置的条件概率。这正是 Transformer 训练阶段就具备的并行能力，在解码时被重新利用起来。

依据 《大语言模型：训练、精调与对齐》第 10.5.1、10.5.2 节

10.6

小结

四种方法分别盯住访存、显存管理、数值精度与解码步数；它们都建立在同一个前提上——自回归解码必须一次一步。

本节内容

10.6 小结

思考题与实践题

推理加速的四条路线

方法	看到的瓶颈	核心手段
Flash Attention	HBM 与 SRAM 间搬运	借 Online-Softmax 做分块计算，中间矩阵不落地
vLLM	KV-Cache 利用率低	分页注意力 + 连续批处理，提升批处理量
SmoothQuant / AWQ	激活值分布难量化	等价变换做平滑，使量化结果逼近原始输出
并行解码	解码串行、一次一词元	把串行解码问题转化为并行验证问题

- 语言模型加速还包括剪枝、蒸馏、稀疏化、局部注意力（如窗口注意力）等方法。
- 系统层面可设计预填充与解码分离的架构；对 MoE 架构与 RNN 类模型，也有各自的加速技术。

推理加速的主线

不重复搬

注意力分块，$\boldsymbol{S}$ 与 $\boldsymbol{P}$ 不再写回 HBM，只留 $O(N)$ 的状态。

不重复占

KV-Cache 按页分配，请求结束即释放；公共前缀只存一份。

不重复存

权重与激活值降到 8 位或 4 位，同样的显存装下更大的模型与批量。

不重复算

一次前向并行验证多个候选词元，把 k 步串行换成 1 步并行。

四种手段可以叠加：分页 KV-Cache 复用 Flash Attention 的内核；投机解码的验证阶段本身就是一次并行前向。

依据 《大语言模型：训练、精调与对齐》第 10.1–10.6 节

思考题与实践题

1 预填充与解码的计算特性有何不同？分别对应哪个性能指标？

2 式 (10-20) 中 $\tilde{l} \neq l_1 + l_2$，请说明差在哪里，并写出正确的合并式。

3 Flash Attention 的复杂度仍是 $O(N^2 d)$，为什么反而更快？省下的是哪一部分开销？

4 V2 把 $\mathrm{diag}(\boldsymbol{L}_i)^{-1}$ 移出内循环，为什么这不改变最终结果？

5 分页注意力为什么能与标准注意力保持数学等价？块表在其中起什么作用？

6 SmoothQuant 的缩放因子为何取几何平均？若只对激活值缩放会怎样？

7 投机解码在全部候选都被拒绝时，输出分布为何仍等于目标分布？

8 实现 Flash Attention V2 的前向与反向算法。

9 实现分页注意力。

10 在块解码基础上进行优化，使得各头输出之间有前后依赖。

11 实现带分页 KV-Cache 管理器的连续批处理服务。

依据 《大语言模型：训练、精调与对齐》第 10.1–10.6 节